

Hands-On-Learning: The Evaluation Breakdown: When Metrics Mislead and How to Fix

Description

In this hands-on lab, you'll investigate why strong automated metric scores don't always translate to high-quality AI-generated summaries. Working as part of a startup's AI team, you'll compute traditional (BLEU, ROUGE) and semantic (BERTScore) metrics, compare them against human judgment, identify specific failure patterns, and design a hybrid evaluation framework that combines automated metrics with strategic human review to catch quality issues that metrics miss.

Learning Objectives

- Compute BLEU, ROUGE, BERTScore, and perplexity metrics using the Hugging Face Evaluate library
- Analyze the relationship between metric scores and actual text quality through comparative evaluation
- Identify specific scenarios where automated metrics fail to capture quality issues
- Design hybrid evaluation frameworks combining automated metrics with human review criteria



Assignment Components

Introductory Slide:

Title: Cracking the Evaluation Paradox: Building AI Quality Assessment Systems That Actually Work

Subtitle: Discover why high BLEU scores hide dangerous errors and design hybrid frameworks that catch what metrics miss

Tools to be used:

- Python 3.8+ with Hugging Face Evaluate library
- BERTScore for semantic similarity measurement
- NLTK for text processing
- Pandas for data analysis and manipulation
- Visual Studio Code (VS Code) with integrated terminal
- CSV editors for data inspection
- JSON tools for framework documentation
- Matplotlib/Seaborn for visualization (optional)

Scenario

You've just joined a promising startup developing an AI-powered summarization tool for enterprise clients. The company's latest model version launched last month with impressive benchmarks—BLEU score of 0.52 and ROUGE-L of 0.61, numbers that would make any AI researcher proud. The engineering team celebrated the launch, confident they'd built a best-in-class product.

But within two weeks, user complaints started flooding in:

- "The summaries miss key details from our financial reports."
- "I found factual errors—the revenue numbers are wrong!"

- "It sounds natural but completely misses the main point"
- "Sometimes it invents information that wasn't in the original."

The VP of Product is puzzled: "Our metrics say this is excellent. What's going on?"
She's tasked you with investigating the evaluation breakdown and designing a better-quality assessment system.

This is a real challenge facing every AI company today. Automated metrics like BLEU and ROUGE are fast and scalable, making them attractive for development.

. But they measure surface-level patterns—word overlap and n-gram matching—not the deeper qualities humans care about: accuracy, completeness, coherence, and trustworthiness.

Companies like OpenAI, Google, and Anthropic invest heavily in "hybrid evaluation frameworks" that combine automated metrics with strategic human review. They've learned the hard way that deploying models based solely on metrics can lead to embarrassing (or dangerous) failures in production.

Your mission has three parts

- 1 Diagnose the problem:** Compute the metrics your team has been using (BLEU, ROUGE, BERTScore) and manually review the actual summaries to understand where metrics diverge from quality
- 2 Identify failure patterns:** Systematically catalog what quality issues the metrics are missing—factual errors, missing information, poor coherence—and understand why automated evaluation fails
- 3 Design a solution:** Create a hybrid evaluation framework that catches quality problems while remaining practical for production deployment

You'll work with real data: 10 news articles with human reference summaries and outputs from three model versions (V1, V2, V3) that have been selected to demonstrate common evaluation challenges. One model scores brilliantly on all metrics but contains dangerous factual errors. Another scores poorly but is actually quite good. Your job is to figure out which is which—and build a system that won't make the same mistakes.

Instructions for learners:

You'll work with three datasets provided:

source_articles.csv - Original news articles (10 articles from business, technology, and health domains)

reference_summaries.csv - Human-written reference summaries for each article

model_summaries.csv - Three model versions (V1, V2, V3) generating summaries for each article

The model summaries have been selected to demonstrate common evaluation challenges:

- High BLEU/ROUGE scores but factual errors
- Semantic similarity but missing key information
- Fluent language but poor coherence
- Word overlap without meaning preservation

ACTIVITY 1: Load Data and Compute Traditional Metrics

You'll start by computing the metrics your startup has been celebrating—BLEU, ROUGE, and perplexity. But as you'll discover, high scores don't always mean high quality. (Refer to repository files for supporting codes).

Step 1: Set up your environment and load the data

- 1 Launch VS Code in your lab environment
- 2 Navigate to `/module2_lab/` in the Explorer panel (left sidebar)
- 3 Open the starter file `evaluation_pipeline_lab.py`
- 4 Verify you can see three CSV data files:

- source_articles.csv (10 original news articles)
- reference_summaries.csv (human-written reference summaries)
- model_summaries.csv (three model versions: V1, V2, V3)

5 Open the integrated terminal (Terminal > New Terminal)

6 Run the setup verification: `python verify_setup.py`

7 Confirm Python 3.8+ and required libraries (evaluate, nltk, bert-score, pandas) are installed

Expected outcome: Terminal displays "✓ Setup verification complete! All dependencies ready." You should see confirmation that evaluate, bert-score, and pandas are properly installed.

Tip: If `verify_setup.py` fails, run: `pip install evaluate nltk bert-score pandas --break-system-packages`

Step 2: Load datasets and examine the summaries

- 1 Locate the section marked `# ACTIVITY 1: TRADITIONAL METRICS` in your starter file
- 2 Run the data loading code
- 3 Review the console output showing number of articles and model versions
- 4 Open `model_summaries.csv` in VS Code to preview the data structure
- 5 Note that each of the 10 articles has 3 model-generated summaries (V1, V2, V3)

Expected outcome: Data loads successfully showing "Articles: 10" and "Model versions: ['V1' 'V2' 'V3']". You understand the data structure with one reference summary per article and three model variants.

What to observe:

- How many articles are in each domain? (business, technology, health)
- What's the typical length of reference summaries vs. model summaries?
- Do you notice obvious differences between models at first glance?

Step 3: Compute BLEU and ROUGE scores for all models

- 1 Navigate to "Practice 1.1" in your code file
- 2 Review the `compute_traditional_metrics()` function
- 3 Run the code to compute metrics for V1, V2, and V3
- 4 Observe the scores printed to terminal for each model
- 5 Note which model has the highest BLEU score
- 7 Check the saved output file: `outputs/traditional_metrics.csv`

Expected outcome: You have BLEU and ROUGE scores for all three models saved to CSV. One model clearly outperforms others on traditional metrics. Take note of the rankings—you'll compare them to actual quality shortly.

What to observe:

- Which model has the highest BLEU score?
- Do ROUGE-1, ROUGE-2, and ROUGE-L rank models consistently?
- What's the score range? (BLEU typically 0.3-0.6 for summaries)

Warning: High BLEU scores look impressive but may be misleading. Keep an open mind about which model is actually best.

Step 4: Read and rate summaries yourself

- 1 Navigate to "Practice 1.2: Manual inspection of summaries"
- 2 Run the code to display one article with all three model summaries
- 3 **Read carefully:** Source excerpt, reference summary, and all three model versions
- 4 For each model summary, mentally rate it 1-5 (1=poor, 5=excellent) based on:
 - Factual accuracy: Does it contain only true information?
 - Completeness: Are key points captured?



Coherence: Does it flow logically?

Naturalness: Does it sound human-written?

5 Write down your ratings in the code comments or on paper

Expected outcome: You've formed your own judgment about which summaries are good/bad. You may already notice a mismatch—does the highest-BLEU model actually produce the best summary to your eyes?

Guiding questions:

- Which summary sounds most natural and trustworthy to you?
- Do any summaries contain specific facts (numbers, dates) that seem questionable?
- Do any miss obvious key details from the source?
- Does your quality ranking match the BLEU/ROUGE ranking?

Tip: Trust your instincts. If a summary "feels off," it probably is—even if metrics say otherwise. This intuition is what we're trying to capture in evaluation systems.

Test Your Work:

- ✓ Traditional metrics CSV created with scores for all models
- ✓ You've identified which model has highest BLEU/ROUGE
- ✓ You've manually read and rated at least one set of summaries
- ✓ You've noticed whether your judgment aligns with metric scores

ACTIVITY 2: Add Semantic Metrics and Compare

Traditional metrics measure word overlap. But what about meaning? BERTScore uses contextual embeddings to capture semantic similarity—theoretically it should be better. Let's test that hypothesis.

Step 1: Compute BERTScore for all models (10 minutes)

- 1 Navigate to # ACTIVITY 2: SEMANTIC METRICS in your code file
- 2 Locate "Practice 2.1: Compute BERTScore"
- 3 Review the `compute_bertscore_batch()` function

- 4 Run the code to compute BERTScore for V1, V2, and V3
- 5 **Be patient:** This computation takes 2-3 minutes as it processes contextual embeddings
- 6 Observe the Precision, Recall, and F1 scores for each model
- 7 Check the merged output: outputs/combined_metrics.csv

Expected outcome: BERTScore computed for all models and merged with traditional metrics. The computation displays progress and completes with "✓ Combined metrics saved." You now have both traditional and semantic metrics for comparison.

What to observe:

- Does BERTScore rank models differently than BLEU?
- Which model has highest BERTScore F1?
- Are there cases where BLEU is high but BERTScore is lower (or vice versa)?

Tip: BERTScore is computationally expensive. If it's too slow, you can reduce `batch_size` or test on fewer articles first (modify the code to slice data: `[:5]` for first 5 articles).

Step 2: Create side-by-side metric comparison

- 1 Navigate to "Practice 2.2: Visual comparison of all metrics"
- 2 Run the code to display all metrics in a clean table format
- 3 Review the "HIGHEST SCORING MODEL PER METRIC" section
- 4 Note which model wins for each metric (BLEU, ROUGE-1, ROUGE-L, BERTScore F1)
- 5 Open outputs/combined_metrics.csv to see the full data

Expected outcome: Clear table showing all metrics side-by-side. You can easily see if different metrics agree or disagree on model rankings.

What to observe:

- Do all metrics agree on the best model?
- Which metric shows the most variation across models?
- Is there a model that consistently ranks high across all metrics?

Step 3: Conduct systematic discrepancy analysis

1 Navigate to "Practice 2.3: Find metric-quality discrepancies"

2 Run the code to examine the first 3 articles in detail

3 For each article, read:

→ Source text excerpt

→ Reference summary

→ All three model summaries (V1, V2, V3)

4 For each article, answer the analysis questions:

- Which summary is best quality to you?
- Does it match the highest BLEU model?
- Does it match the highest BERTScore model?
- What specific quality issues do you notice that metrics missed?

5 Document your findings in the code comments or in a notes file

Expected outcome: You've systematically compared 3 articles and identified at least one case where metrics and human judgment diverge. You can articulate specific quality issues (e.g., "V3 has wrong revenue number" or "V1 misses key policy details").

Guiding questions:

- When BLEU and BERTScore disagree, which aligns better with your judgment?
- Can you find a summary with HIGH scores but OBVIOUS quality problems?
- Can you find a summary with LOWER scores but BETTER actual quality?
- What types of errors do metrics fail to catch? Examples:
 - Factual errors (wrong numbers, dates, names)
 - Missing critical information
 - Incoherent or illogical flow
 - Hallucinated details not in source

Hallucinated details not in source

Tone or style mismatches

Critical insight: This is the evaluation paradox in action. If you find summaries that score well but have quality issues, you've discovered why companies can't rely on metrics alone.

Test Your Work:

✓ BERTScore computed and saved for all models

✓ Combined metrics CSV created

✓ You've examined at least 3 articles systematically

- 
- ✓ You've identified at least one metric-quality discrepancy
 - ✓ You can name specific quality issues that metrics missed

ACTIVITY 3: Identify Metric Failures and Design Hybrid Evaluation Framework

Now for the detective work: systematically catalog where metrics fail, then design an evaluation system that combines automation with human insight to catch what matters.

Step 1: Create systematic failure analysis

- 1 Navigate to # ACTIVITY 3: METRIC FAILURE ANALYSIS in your code
- 2 Locate "Practice 3.1: Systematic failure analysis"
- 3 Run the code to generate the failure analysis template CSV
- 4 Open outputs/failure_analysis_template.csv in VS Code or Excel
- 5 For each summary (30 total: 10 articles $\times$ 3 models), fill in:

- has_factual_error: True if contains wrong facts (numbers, dates, names)
- missing_key_info: True if omits important points from source
- poor_coherence: True if logic doesn't flow or has contradictions
- unnatural_language: True if sounds robotic or awkward
- overall_quality: Rate 1-5 (1=poor, 5=excellent)
- notes: Specific observations about problems

6 Save your completed analysis as outputs/failure_analysis_completed.csv

Expected outcome: Completed failure analysis for all 30 summaries documenting specific quality issues. This systematic review takes 10-15 minutes but provides the data foundation for understanding failure patterns.

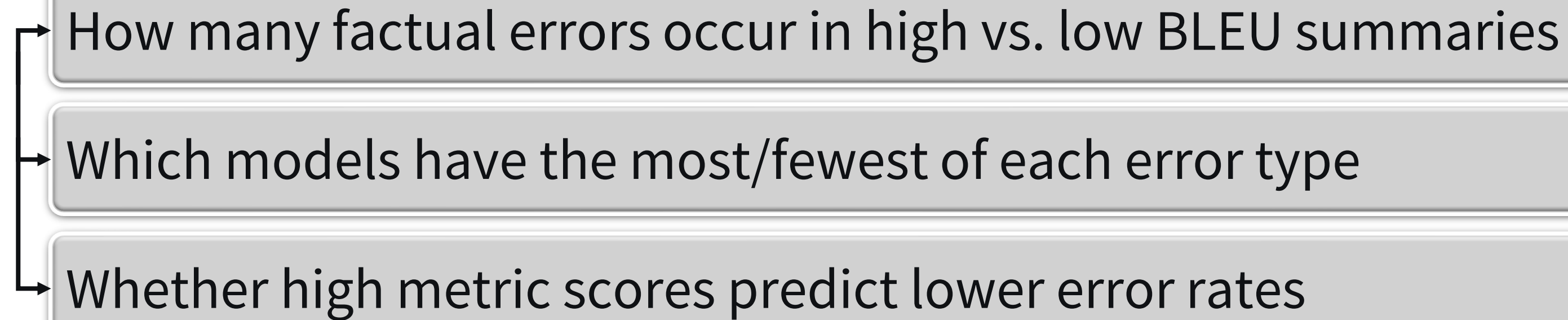
What to do:

- Read each summary against its source article carefully
- Check facts: Do numbers, dates, and names match the source?
- Check completeness: Are the main points captured?
- Check coherence: Does the summary make logical sense?
- Be honest: If you're unsure, mark it and note why

Tip: Focus on factual accuracy first—it's the most critical dimension and the one metrics miss most often. If a number or date is wrong, that's a hard failure regardless of how well the summary flows.

Step 2: Analyze failure patterns and correlations

- 1 Navigate to "Practice 3.2: Correlate manual assessments with metrics"
- 2 Ensure your failure_analysis_completed.csv is saved
- 3 Run the correlation code to analyze relationships
- 4 Review the output showing:

- 
- How many factual errors occur in high vs. low BLEU summaries
 - Which models have the most/fewest of each error type
 - Whether high metric scores predict lower error rates

5 Take notes on key patterns you discover

Expected outcome: Clear patterns emerge showing that metrics don't predict critical failures. For example, you might find: "Model V3 has highest BLEU but 60% of its summaries contain factual errors."

What to observe:

- Do high-scoring models have fewer errors? (Probably not!)
- Which error type is most common overall?
- Is there a model that's low-scoring but actually accurate?
- Which quality dimension do metrics predict best/worst?

Critical finding: If high BLEU doesn't correlate with low factual errors, that proves BLEU can't be used alone for quality assessment.

Step 3: Design your hybrid evaluation framework

1 Navigate to "Practice 3.3: Design a hybrid evaluation framework"

2 Review the template framework provided in the code

3 **Customize it** based on your failure analysis findings:

→ Adjust metric weights based on what you found (e.g., if BERTScore better predicts quality, give it a higher weight)

→ Set thresholds that would catch the failures you identified

→ Add human review triggers for the error types you found

Define what counts as a deployment blocker (e.g., any factual error)

4 Update the reasoning fields to justify your choices

5 Run the code to save your framework as outputs/hybrid_framework.json

Expected outcome: **Deliverable 4 complete** - A customized hybrid framework JSON file that reflects your findings and includes a clear rationale for design choices.

Your framework should include:

- Automated metrics section with weights (total to 1.0 or 100%)
- Specific thresholds that trigger human review
- List of quality checkpoints for human reviewers
- Deployment gates (what scores/conditions allow auto-approval vs. require review)
- Justification for each major design decision

Framework design principles:

→ **High-stakes dimensions get higher weights:** If factuality is critical, weight it heavily

→ **Strategic triggers prevent errors:** Flag low-scoring summaries AND specific risk patterns

→ **Sustainable review load:** Aim for 15-25% flagged for review (realistic for teams)

→ **Clear deployment gates:** Define what auto-approves, what needs review, and what auto-rejects

Step 4: Test your framework on sample data

1 Navigate to "Practice 3.4: Apply your framework to test cases"

2 Run the code to test your framework on sample summaries

3 Review the output showing which summaries would be:

→ APPROVED (auto-deployed)

→ PENDING_REVIEW (flagged for human check)

→ REJECTED (blocked from deployment)

4 Verify your framework catches the quality issues you identified earlier

5 Adjust thresholds if needed and re-test

Expected outcome: Your framework successfully flags problematic summaries for review while auto-approving high-quality ones. You can explain why each decision makes sense.

What to validate

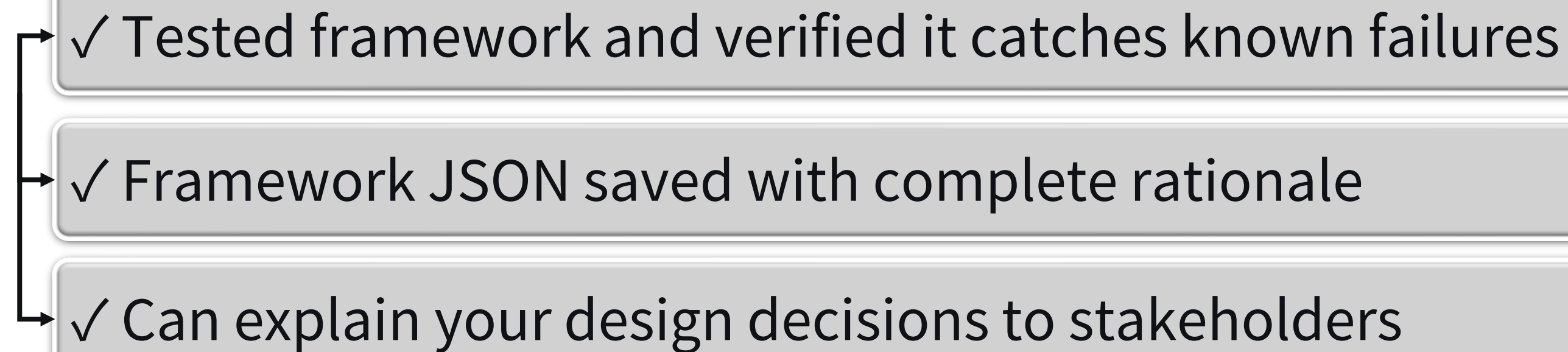
- Does your framework flag summaries with factual errors?
- Does it approve the genuinely good summaries?
- Is the review rate realistic (15-30% is typical)?

Would this catch the user complaints from the scenario?

Tip: There's no single "correct" framework. The best design depends on your domain, risk tolerance, and team capacity. A medical summarization system needs different thresholds than a creative marketing tool.

Test Your Work:

- ✓ Completed failure analysis for all 30 summaries
- ✓ Identified clear patterns in what metrics miss
- ✓ Designed a hybrid framework with justified weights and thresholds

- 
- ✓ Tested framework and verified it catches known failures
 - ✓ Framework JSON saved with complete rationale
 - ✓ Can explain your design decisions to stakeholders

Deliverable for Submission (AI Grading)

1 Deliverable 1: Computed Metrics Analysis Report

Question Prompt: "Submit a structured report containing BLEU, ROUGE, and BERTScore metrics for all three model versions (V1, V2, V3), along with your analysis of which model ranks highest for each metric. Include a brief comparison (150-200 words) discussing whether different metrics agree on model rankings and any notable discrepancies you observed. Your report should demonstrate understanding of what each metric measures."

Response Type: File Upload

File Types: Text (.csv, .txt, .md), Coding (.py for annotated code), Audio (.mp3, .wav for verbal explanation), Video (.mp4, .mov for presentation)

File Name: computed_metrics_analysis (with appropriate extension)

Grading Criteria:

- Contains complete metrics for all three models (BLEU, ROUGE, BERTScore) - 5 pts
- Correctly identifies the highest-scoring model for each metric - 3 pts
- Includes comparison analysis (150-200 words) - 4 pts
- Demonstrates understanding of metric differences - 3 pts

2 Deliverable 2: Manual Quality Assessment Documentation

Question Prompt: "Submit your completed manual quality assessment analyzing at least 5 summaries (across different models and articles). For each summary, document: (1) whether it contains factual errors, (2) whether key information is missing, (3) coherence issues, (4) naturalness problems, and (5) overall quality rating (1-5). Include specific examples of quality issues you identified that automated metrics failed to detect. Your documentation should clearly show a systematic evaluation comparing human judgment with metric scores."

Response Type: File Upload

File Types: Text (.csv, .txt, .md), Audio (.mp3, .wav, .aac for verbal analysis), Video (.mp4, .mov for walkthrough)

File Name: manual_quality_assessment (with appropriate extension)

Grading Criteria:

- Assesses at least 5 summaries systematically - 5 pts
- Documents all required quality dimensions (factual, completeness, coherence, naturalness, overall) - 8 pts
- Provides specific examples of issues metrics missed - 7 pts
- Shows clear comparison between human judgment and metric scores - 5 pts

3 Deliverable 3: Metric Failure Pattern Analysis

Question Prompt: "Submit a comprehensive analysis (300-400 words or 3–5-minute audio/video) identifying and explaining specific patterns where automated metrics fail to capture quality issues. Address the following: (1) What types of errors do BLEU/ROUGE/BERTScore consistently miss? (2) Provide at least three concrete examples from your data showing high metric scores paired with quality problems. (3) Explain WHY these metrics fail for these error types—what aspect of quality are they unable to measure? Your analysis should demonstrate deep understanding of the evaluation paradox."

Response Type: File Upload

File Types: Text (.txt, .md), Audio (.mp3, .wav, .aac, .flac), Video (.mp4, .mov, .webm - max 15 min)

File Name: metric_failure_analysis (with appropriate extension)

Grading Criteria:

- Identifies specific error types metrics miss (factual errors, missing info, etc.) - 8 pts
- Provides at least 3 concrete examples with details - 9 pts
- Explains WHY metrics fail for these cases - 8 pts
- Meets word/time requirements (300-400 words or 3-5 min) - 5 pts

4 Deliverable 4: Hybrid Evaluation Framework Design

Question Prompt: "Submit your designed hybrid evaluation framework in JSON format (or equivalent structured document) that combines automated metrics with strategic human review. Your framework must include: (1) Automated metrics section with specific weights for BLEU/ROUGE/BERTScore totaling 100%, (2) Clear thresholds that trigger human review, (3) List of manual quality checkpoints for reviewers, (4) Deployment gates defining auto-approve/review/reject criteria, and (5) Detailed rationale (200-300 words) justifying your design choices based on your failure pattern analysis. The framework should be practical for production deployment."

Response Type: File Upload

File Types: Text (.json, .txt, .md, .csv), Audio (.mp3, .wav for framework walkthrough), Video (.mp4, .mov for presentation)

File Name: hybrid_evaluation_framework (with appropriate extension)

Grading Criteria:

- Includes all required components (metrics with weights, thresholds, checkpoints, gates) - 10 pts
- Weights are justified and total to 100% - 5 pts
- Human review triggers are specific and strategic - 5 pts
- Includes detailed rationale (200-300 words) explaining design decisions - 7 pts

Framework is practical and deployable (realistic review loads, clear criteria) - 3 pts

Additional Information

Best Practices for AI-Graded Submission

Creating Effective Question Prompts

- 1** Be Specific: Include specific numbers or criteria in your prompts to help AI generate clear assessment criteria.
- 2** Include Context: Reference specific elements from the HOL activity.
- 3** Encourage Depth: Ask for examples, justifications, or connections.

4 Align with Objectives: Ensure prompts assess stated learning objectives.

Submission Requirement Guidelines

1 Clarity: Each requirement should have one clear focus.

2 Balance: Mix different types of submissions (creative, analytical, reflective).

3 Inclusivity: Always provide alternative formats.

3 Progression: Order requirements from concrete to abstract.

File Type Selection Guide

Always offer multiple formats for inclusivity:

Submission Type	Recommended File Options
Written Work	Text, Audio (for verbal responses), Video (for sign language)
Visual Creation	Image, Video, Coding (for programmatic art)
Technical Solution	Coding, Text, Video (for demonstration)
Presentation	Video, Audio, Image (slides), Text (script)
Reflection	Text, Audio, Video

Standard Options:

- Text: .txt, .text, .csv, .md, .json
- Image: .png, .jpg, .jpeg, .jfif, .webp, .bmp, .tiff, .tif
- Video: .mp4, .mov, .webm
- Audio: .mp3, .wav, .aac, .flac
- Coding: .bash, .bat, .c, .cpp, .css, .go, .htm, .html, .java, .json, .jsp, .jsx, .md, .php, .py, .r, .scss, .sh, .sml, .sql, .swift, .ts, .tsx, .xml, .yaml, .js

Size Limits:

- Images: 5 MB maximum
- Files: 2 GB maximum
- Videos: 15 minutes maximum

Accessibility Considerations:

- Always allow multiple file types for each requirement.
- Include both video AND audio options for verbal responses.
- Provide text alternatives for visual submissions.

Quality Assurance Checklist

Before finalizing your HOL:

- ✓ All prompts are clear and specific
- ✓ Multiple file types offered for each upload requirement
- ✓ Submission types match the nature of the task
- ✓ Requirements align with learning objectives
- ✓ Instructions are step-by-step and actionable
- ✓ Accessibility options are available throughout

- ✓ Time estimates are realistic and add up to stated duration (90 min total)
- ✓ Expected outcomes are provided for each major step
- ✓ Tips and warnings are included where necessary

APPENDIX: Supporting Materials

Common Issues & Solutions

Problem: BERTScore computation is very slow. Solution: This is normal—BERTScore uses large neural models. Reduce batch_size from 8 to 4, or compute for fewer articles first (slice data to first 5 articles). If a GPU is available, change device='cpu' to device='cuda' for 10x speedup.

Problem: Metrics seem to contradict each other completely. Solution: This is expected and is actually the core learning opportunity! Document the contradictions carefully. Ask yourself: "What quality dimension does each metric capture?" BLEU measures exact word overlap, ROUGE measures content recall, BERTScore measures semantic similarity—they can easily disagree.

Problem: Can't decide if a summary has factual errors Solution: Check every specific claim (numbers, dates, names, facts) against the source article word-by-word. If information appears in the summary but NOT in the source, it's likely hallucinated. When in doubt, flag it as "potential error" and note your uncertainty.

Problem: Unsure what weights to assign in hybrid framework Solution: Start with equal weights, then adjust based on your priorities and findings. If you found many factual errors that metrics missed, increase the weight on human review for factuality. If your domain is high-stakes (medical, legal, financial), weight accuracy heavily. If creative/marketing, weight fluency higher.

Problem: Framework flags too many or too few summaries for review Solution: Adjust thresholds iteratively. If 50% are flagged (too many), raise thresholds. If only 5% are flagged (too few), lower thresholds. Aim for 15-25% flagged—this is realistic for human review capacity while catching most issues.

Problem: Import errors with evaluate library Solution: Run `pip install --upgrade evaluate datasets --break-system-packages`. Verify installation: `python -c "import evaluate; print(evaluate.__version__)"`. If still failing, try: `pip uninstall evaluate` then `pip install evaluate==0.4.0 --break-system-packages` for a specific stable version.

Problem: NLTK download fails with SSL error. Solution: Run in Python: `import ssl; ssl._create_default_https_context = ssl._create_unverified_context`, then `import nltk; nltk.download('punkt')`. Or manually download punkt tokenizer from NLTK website.

Problem: CSV file won't open or has formatting issues. Solution: Try different CSV readers (VS Code, Excel, Google Sheets). If special characters appear garbled, open with UTF-8 encoding. If commas in text cause issues, use a different delimiter like semicolons.

Key Takeaways

Evaluation Paradox:

- High automated metric scores $\neq$, High actual quality
- Metrics optimize for surface patterns (word overlap, semantic similarity)
- Quality requires understanding (facts, completeness, appropriateness)

This gap is the fundamental challenge in AI evaluation

Metric Limitations:

→ **BLEU:** Measures exact n-gram overlap. Misses paraphrasing, semantic equivalence, and all aspects of meaning beyond word matching. Cannot detect factual errors if wrong words match reference patterns.

→ **ROUGE:** Measures content recall via word/n-gram overlap. Better than BLEU for summaries but still surface-level. Misses factual accuracy, coherence, and appropriateness.

BERTScore: Measures semantic similarity using contextual embeddings.

Better captures meaning but still cannot verify facts, detect hallucinations, or judge task appropriateness.

All three together: Still miss critical dimensions like factual accuracy, logical coherence, completeness, and domain-specific quality requirements.

Metric Limitations:

- 1 They measure similarity to references, not quality in absolute terms
- 2 They cannot verify factual accuracy—" \$52 billion" scores similarly to "\$45 billion"

- 3 They reward fluency even when content is wrong
- 4 They miss logical contradictions and incoherence
- 5 They can't judge task-appropriateness or user needs
- 6 They have no world knowledge to check claims

Hybrid Evaluation Principles:

- Combine automation (speed, consistency, scalability) with human insight (accuracy, nuance, judgment)
- Use metrics as triggers, not verdicts—low scores flag for review
- Focus human effort where it provides maximum value (fact-checking, completeness)
- Set realistic review loads (15-30% is sustainable for most teams)
- Continuous calibration: track metric-human correlation over time
- Domain-specific weights: medical, ≠ creative, ≠ financial

Framework Design Considerations:

→ **High-stakes domains** (medical, legal, financial): High factuality weight (60%+), zero tolerance for errors, require expert review

→ **Medium-stakes** (news, business): Balanced weights, strategic review triggers, focus on accuracy and completeness

→ **Low-stakes** (creative, marketing): Higher fluency/engagement weights, lighter review, focus on appropriateness and tone

Production Deployment:

- Never rely on a single metric—use multiple complementary measures
- Build feedback loops from user complaints to improve evaluation
- Track false positive (good content rejected) and false negative (bad content approved) rates
- Regular calibration with expert reviewers (monthly recommended)
- Version control your evaluation frameworks—they're as important as models
- Monitor metric-quality correlation over time—it may degrade as models evolve

Critical Insight: The goal isn't finding the perfect metric—it's building a system that combines automated efficiency with human judgment to catch what matters most for YOUR specific application and users.

Metrics are tools for filtering and prioritization, not final arbiters of quality.

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.